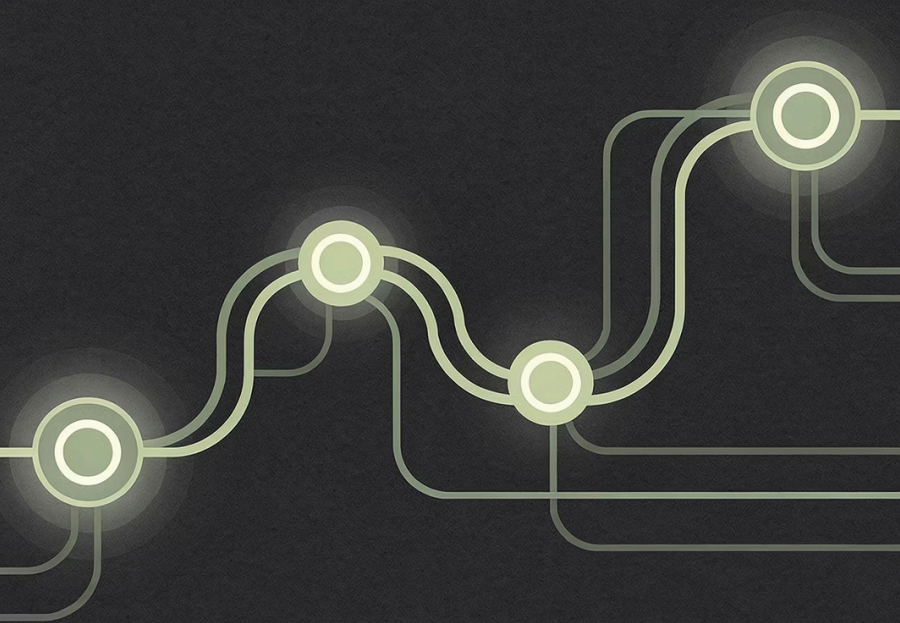




UNLOX

Data Quality & Feature Engineering

Building better models starts with better data. A practical guide to cleaning, transforming, and engineering the features that power machine learning.

Why Data Quality Is Everything

The quality of an AI model is **directly dependent** on the quality of its training data. No amount of hyperparameter tuning or architectural innovation can compensate for a dataset riddled with errors, inconsistencies, and inaccuracies.

Data cleaning is the critical first step in any ML pipeline — the process of systematically identifying and correcting problems in your dataset before training begins. Think of it as laying a solid foundation before constructing a building.

- ❏ Industry studies estimate that data scientists spend **60–80%** of their time on data preparation and cleaning, making it the most labor-intensive phase of any ML project.



Handling Missing Data

Missing values are one of the most common problems in real-world datasets. Sensors fail, users skip form fields, and records get corrupted. Before training a model, you need a deliberate strategy for dealing with these gaps. Here are the primary approaches:

1

Deletion

Remove rows (listwise) or columns containing missing values. This is the simplest approach, but it risks losing valuable data — especially when missingness is widespread across the dataset.

2

Mean / Median Imputation

Replace missing values with the column's **mean** (for normally distributed data) or **median** (for skewed distributions). Quick and effective, but it reduces variance and can distort relationships between features.

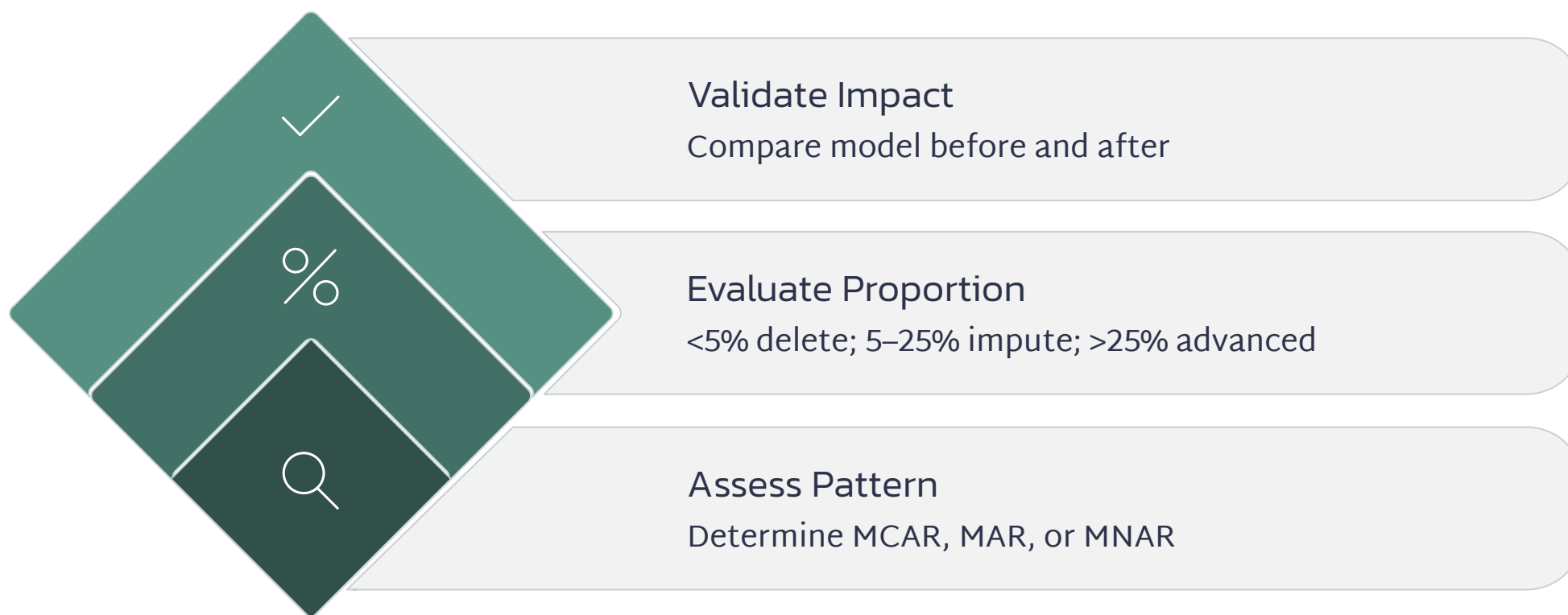
3

Advanced Imputation

Use model-based methods like **K-Nearest Neighbors** or **iterative imputation** to predict missing values from other features. More accurate, but computationally expensive and harder to implement.

Choosing the Right Strategy for Missing Data

Not all missing data is created equal. The right approach depends on **how much** data is missing and **why** it's missing. Understanding the mechanism behind missingness is key to making an informed decision.



Always validate your chosen strategy by comparing model performance metrics before and after applying your missing data treatment. The goal is to preserve as much signal as possible while minimizing introduced bias.

Handling Duplicate Records

Why Duplicates Are Dangerous

Duplicate records silently bias your model's training process. When the same observation appears multiple times, the model over-weights those patterns — leading to overfitting and poor generalization on unseen data.

Common sources of duplicates include:

- Multiple form submissions by the same user
- Merging overlapping data sources
- ETL pipeline errors or retries
- Web scraping the same page twice

Detection & Removal Techniques

- Exact Duplicates

Rows where every column value is identical. Use `df.duplicated()` in pandas to flag them, then drop with `df.drop_duplicates()`.

- Near Duplicates

Rows that represent the same entity but differ slightly (e.g., "Jon Smith" vs. "John Smith"). Use fuzzy matching libraries like `fuzzywuzzy` or record linkage techniques.

- Subset Duplicates

Check for duplicates based on key columns only (e.g., same user ID and timestamp). Use `df.duplicated(subset=[...])` for targeted detection.

UNLOX

QUICK REFERENCE

Data Cleaning Cheat Sheet

Keep this reference handy as you work through your data preparation pipeline. Each issue type has preferred strategies depending on context and dataset size.

Issue	Detection Method	Recommended Fix	Risk if Ignored
Missing values	<code>.isnull().sum()</code>	Imputation or deletion	Biased predictions
Exact duplicates	<code>.duplicated()</code>	Drop duplicates	Overfitting
Near duplicates	Fuzzy matching	Record linkage & merge	Inflated metrics
Outliers	Z-score or IQR	Cap, transform, or remove	Skewed model weights
Inconsistent formats	Value counts inspection	Standardize & normalize	Feature fragmentation

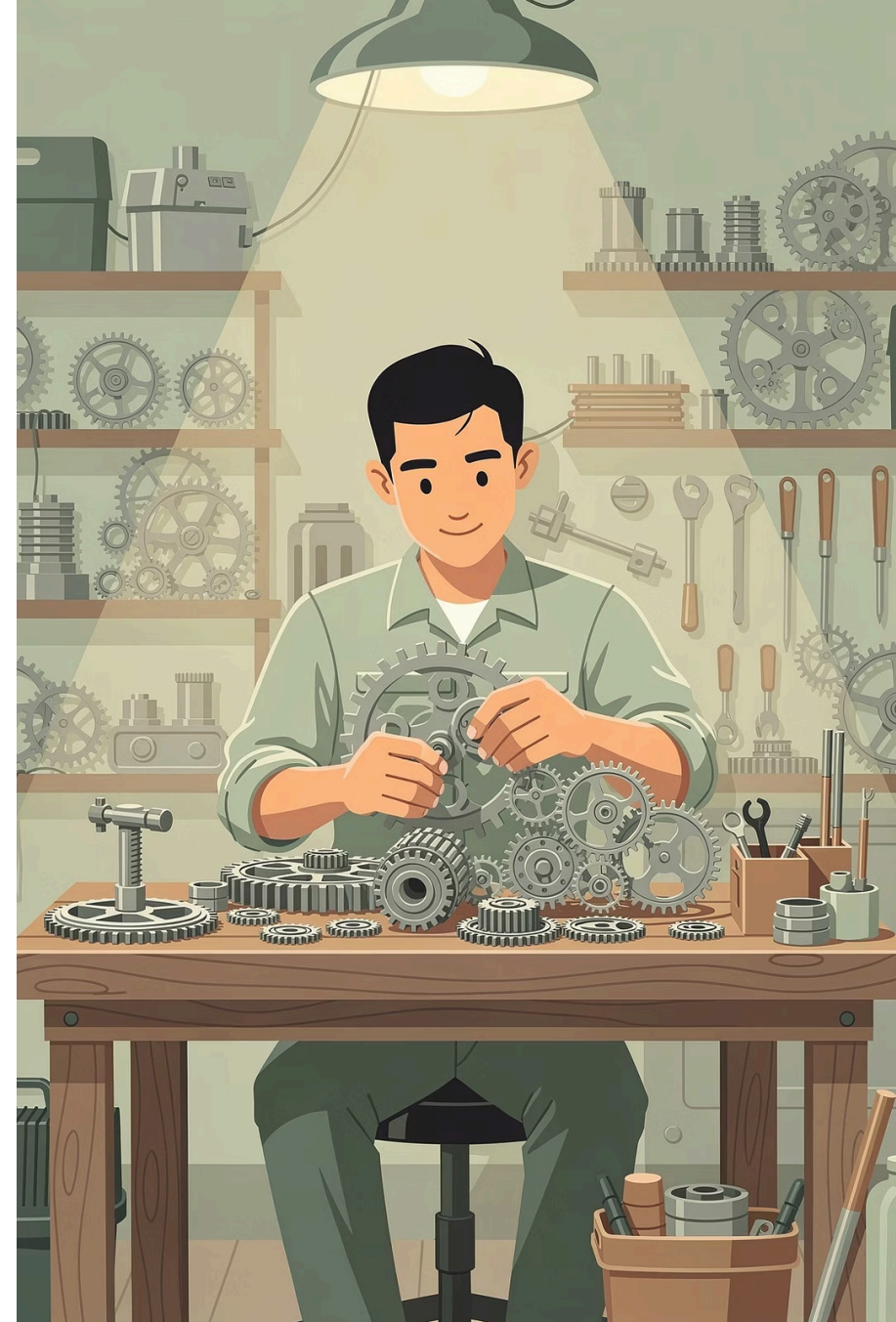
UNLOX

CHAPTER 5.2

Feature Engineering

Feature engineering is the art and science of using domain knowledge to create new, more informative input variables from raw data. It is widely considered the single most impactful lever for improving model performance — often more so than algorithm selection itself.

Great features make simple models powerful. Poor features make complex models useless.



UNLOCK

FEATURE ENGINEERING

Why Feature Engineering Matters

Raw data rarely comes in a form that's optimal for machine learning. Feature engineering bridges the gap between what you *have* and what your model *needs*. Here's why it's so critical:



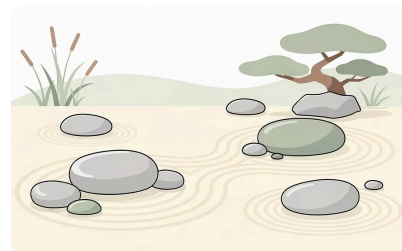
Reveals Hidden Patterns

Combining or transforming existing features can expose relationships that raw variables alone can't express — like deriving "age of account" from a signup date.



Boosts Model Accuracy

Well-engineered features can dramatically improve accuracy, precision, and recall without changing the underlying algorithm or adding more data.



Simplifies Complexity

Good features allow simpler, more interpretable models to achieve strong results — reducing training time, overfitting risk, and computational cost.

Feature Scaling: Bringing Data to a Common Scale

Many machine learning algorithms — including gradient descent-based models, SVMs, and K-Nearest Neighbors — are **sensitive to the scale** of input features. If one feature ranges from 0 to 1 and another from 0 to 100,000, the model will disproportionately weight the larger feature. Scaling corrects this imbalance.

Min-Max Scaling

Rescales every feature to a fixed range, typically **[0, 1]**. Formula: $(x - \min) / (\max - \min)$. Best when you know the data has a bounded range and no significant outliers.

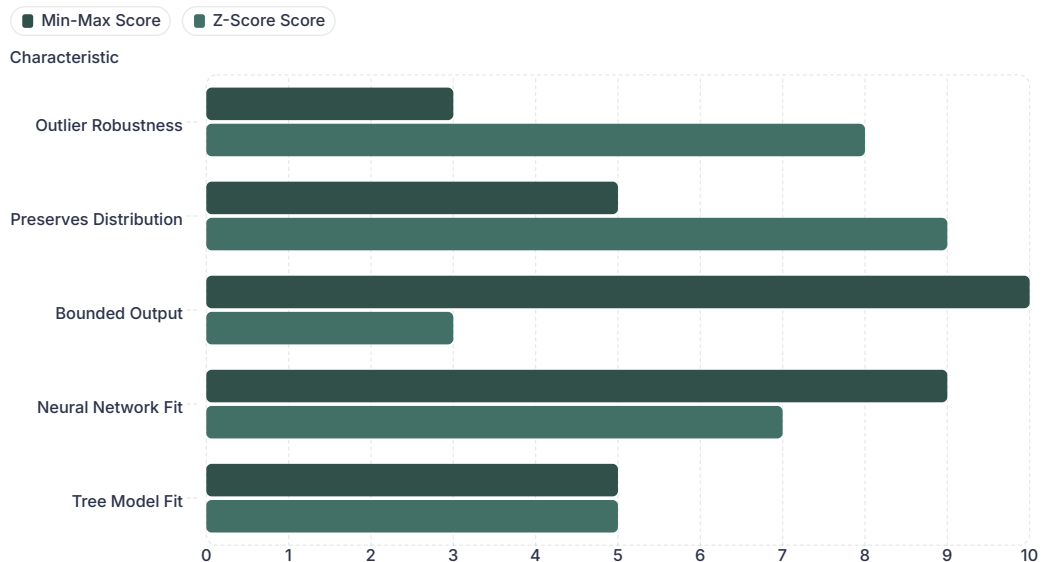
Standardization (Z-Score)

Centers data around a mean of **0** with a standard deviation of **1**. Formula: $(x - \mu) / \sigma$. Preferred when data contains outliers or follows a roughly Gaussian distribution.

📌 **Pro tip:** Always fit your scaler on the **training set only**, then transform both training and test sets. Fitting on the entire dataset introduces data leakage and inflates your evaluation metrics.

Min-Max vs. Standardization: When to Use Each

Choosing the right scaling technique depends on your data characteristics and the algorithm you're using. Here's a side-by-side comparison to guide your decision:



Key Takeaways

- Use **Min-Max** for neural networks and when you need bounded [0,1] outputs
- Use **Z-Score** when outliers are present or data is roughly Gaussian
- Tree-based models (Random Forest, XGBoost) are generally **scale-invariant** — scaling is optional

Scores represent relative suitability on a 1–10 scale for each characteristic.

UNLOCK

PUTTING IT ALL TOGETHER

The Complete Data Preparation Pipeline

Data quality and feature engineering are not isolated steps — they form a **sequential pipeline** where each stage builds on the last. Following a consistent order ensures reproducibility and prevents common pitfalls like data leakage.



Explore

Profile your data. Check distributions, types, and missingness patterns.



Clean

Handle missing values, remove duplicates, fix inconsistencies.



Engineer

Create new features, encode categoricals, and apply domain knowledge.



Scale

Normalize or standardize features. Fit on train, transform both sets.



UNLOCK

KEY TAKEAWAYS

Key Takeaways

Garbage In, Garbage Out

No model can overcome poor data. Invest time in thorough cleaning before training.

Choose Strategy Deliberately

Whether deleting, imputing, or deduplicating — always understand the trade-offs of your approach.

Features > Algorithms

Well-crafted features often outperform complex models trained on raw data.

Scale With Intention

Match your scaling technique to your data characteristics and chosen algorithm.